SHERLOCK AI

AI Security Review For Aave - BGD Labs

Sherlock AI Audit Prepared For: **Aave - BGD Labs**
Date Audited: **March 5 - March 26, 2026**

Introduction

Introduction

Scope

Repository: aave-dao/aave-v3-origin

Audited Commit: 75145ad021c123a14d9190c67bd30e6867e22cb7

Findings

Each issue has an assigned severity:

- High issues are directly exploitable security vulnerabilities that need to be fixed.
- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
0	0	4

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Issue L-1: LiquidationLogic.executeLiquidationCall: Double updateState on same reserve when collateralAsset == debtAsset leads to treasury over-accrual diluting aToken holders

This issue has been acknowledged by the team and fixed at commit `e357e12`.

Description

The `LiquidationLogic.executeLiquidationCall` function updates the state of a reserve twice when the collateral asset is the same as the debt asset, resulting in an over-accrual to the treasury. This occurs because both `debtReserveCache` and `collateralReserveCache` are initialized from the same storage, creating two separate memory structures with identical `reserveLastUpdateTimestamp`. When `debtReserve.updateState(debtReserveCache)` is called, it updates the storage `lastUpdateTimestamp` to `block.timestamp` and modifies `debtReserveCache.reserveLastUpdateTimestamp`. However, `collateralReserveCache.reserveLastUpdateTimestamp` remains unchanged. Consequently, when `collateralReserve.updateState(collateralReserveCache)` is executed, it passes the staleness check and runs `_accrueToTreasury` a second time. While `_updateIndexes` is idempotent, `_accrueToTreasury` uses an addition operation on `reserve.accruedToTreasury`, doubling the treasury accrual for the period since the last update.

Attack Path

The attack path involves a scenario where a user supplies and borrows the same asset, and a liquidator triggers the double accrual:

1. A user supplies asset X and uses it as collateral while also borrowing asset X. The reserve has not been updated for some time, allowing meaningful interest to accrue.
2. The user's health factor drops below 1 due to interest accrual or price changes.
3. A liquidator calls `Pool.liquidationCall` with `collateralAsset` and `debtAsset` both set to X.
4. In `executeLiquidationCall`, both `debtReserveCache` and `collateralReserveCache` are cached from the same reserve storage.
5. `debtReserve.updateState(debtReserveCache)` runs `_accrueToTreasury`, adding an amount to `reserve.accruedToTreasury`.
6. `collateralReserve.updateState(collateralReserveCache)` runs again because `collateralReserveCache.reserveLastUpdateTimestamp` is still the old timestamp, adding the same amount to `reserve.accruedToTreasury` again.

7. As a result, `reserve.accruedToTreasury` is now twice the correct value for this accrual period.

Impact

This vulnerability results in the treasury receiving double the intended accrual amount, diluting the value of aTokens held by users. The over-accrual to the treasury reduces the yield available to aToken holders, effectively transferring value from users to the treasury. This can lead to a significant loss of yield for users, especially in reserves with high activity and interest accrual. The issue persists until the reserve is updated again, potentially affecting multiple users and transactions.

Suggested Fix

Skip the second reserve update when debtAsset equals collateralAsset to prevent double accrual.

```
-    ReserveData storage collateralReserve =
 ↪  pool.getReserveData(params.collateralAsset);
-    ReserveCache memory collateralReserveCache = collateralReserve.cache();
-    collateralReserve.updateState(collateralReserveCache);
+    ReserveData storage collateralReserve =
 ↪  pool.getReserveData(params.collateralAsset);
+    ReserveCache memory collateralReserveCache = collateralReserve.cache();
+    if (params.collateralAsset == params.debtAsset) {
+      // reuse updated cache to avoid double accrual
+      collateralReserveCache = debtReserveCache;
+    } else {
+      collateralReserve.updateState(collateralReserveCache);
+    }
```

Issue L-2: EModeEngine._findFirstUnusedEmodeCategory: uint8 Loop Overflow Causes Panic Instead of Custom Error

This issue has been acknowledged by the team but won't be fixed at this time.

Description

The `EModeEngine._findFirstUnusedEmodeCategory` function is designed to find the first unused eMode category by iterating through potential category IDs. It uses a `for` loop with a `uint8` counter, iterating from 1 to 255. However, when the counter reaches 255 and no unused category is found, the increment operation `i++` causes an overflow in Solidity 0.8.x's checked arithmetic, resulting in a panic revert with code 0x11. This prevents the function from reaching the intended custom error `revert NoAvailableEmodeCategory()`, which is supposed to indicate that no available category was found. The condition `i < 256` is always true for a `uint8` type, making the custom error unreachable.

Attack Path

Consider a scenario where all 255 eMode categories (IDs 1-255) are already in use, each having a non-zero `liquidationThreshold`. A governance payload attempts to create a new eMode category by calling `createEModeCategories`. This triggers the `EModeEngine._findFirstUnusedEmodeCategory` function, which loops through the category IDs from 1 to 255 without finding an unused slot. When the loop counter reaches 255, the loop body completes without returning, and the increment operation `i++` causes an overflow. Consequently, the transaction reverts with an arithmetic overflow panic (0x11) instead of the custom `NoAvailableEmodeCategory()` error.

Impact

The impact of this issue is primarily a user experience concern rather than a security vulnerability. The transaction still correctly reverts, preventing any invalid state from being reached. However, the revert reason is a generic panic code instead of the more informative custom error. This could lead to confusion for developers or users trying to diagnose the issue. The precondition for this scenario–having all 255 eMode categories occupied–is extremely unlikely in any real deployment, further limiting the practical impact of this issue.

Suggested Fix

Use a wider loop counter to avoid overflow and reach the custom error.

```
-    for (uint8 i = 1; i < 256; i++) {
+    for (uint256 i = 1; i < 256; i++) {
```

Issue L-3: LiquidationLogic.executeLiquidationCall: hasNoCollateralLeft is false for multi-collateral users with dust in other reserves, preventing bad debt burning even when effective collateral is zero

This issue has been acknowledged by the team and fixed at commit `75145ad`.

Description

The `LiquidationLogic.executeLiquidationCall` function fails to correctly identify when a borrower has no collateral left if the borrower holds a small, dust-valued collateral position in a secondary reserve. This occurs because the `hasNoCollateralLeft` flag is computed using a strict equality check between `totalCollateralInBaseCurrency` and `collateralToLiquidateInBaseCurrency`. The `totalCollateralInBaseCurrency` sums collateral across all reserves, while `collateralToLiquidateInBaseCurrency` only considers the specific collateral being liquidated. If a borrower has a trivial amount of collateral in another reserve, such as 1 wei of a high-decimal token, this can cause the equality check to fail. As a result, the `_burnBadDebt` function is not called, leaving the borrower with unpayable debt and a stale position. The dust collateral in the other reserve cannot be economically liquidated, creating a potential griefing vector where users can prevent bad debt cleanup by maintaining trivial collateral in multiple reserves.

Attack Path

Consider a scenario where a user supplies 1000 USDC as primary collateral and 1 wei of WETH as secondary collateral, both enabled as collateral. The user borrows against the combined collateral, and a price movement causes the health factor to drop below 1. A liquidator then liquidates all USDC collateral, resulting in a zero USDC balance. The `collateralToLiquidateInBaseCurrency` equals the full USDC collateral value in base currency. However, the `totalCollateralInBaseCurrency` includes both the USDC value and the WETH dust value. Even if the WETH dust contributes just 1 unit (e.g., 1e-8 USD with 8-decimal base), the `totalCollateralInBaseCurrency` exceeds the `collateralToLiquidateInBaseCurrency`. Consequently, `hasNoCollateralLeft` is false, and `_burnBadDebt` is not called. The remaining debt across all reserves is left unaddressed. The WETH dust collateral is too small to be economically liquidated, but the user still has outstanding debt that will not become a deficit.

Impact

This issue creates a griefing vector where users can prevent the cleanup of bad debt by maintaining trivial collateral in multiple reserves. The protocol is unable to convert stale debt to a deficit, leaving the borrower's debt unaddressed. While a second liquidation

targeting the dust collateral is technically possible, it is economically unprofitable, as the liquidator would receive dust collateral worth fractions of a cent while incurring significant gas costs. As a result, no rational liquidator would perform this action, leaving the protocol with unpayable debt and a permanent stale position.

Suggested Fix

Treat negligible residual collateral as none when setting hasNoCollateralLeft during liquidation.

```
-    bool hasNoCollateralLeft = totalCollateralInBaseCurrency ==
 ↪  collateralToLiquidateInBaseCurrency;
+    bool hasNoCollateralLeft =
+      totalCollateralInBaseCurrency <= collateralToLiquidateInBaseCurrency + 1;
```

Issue L-4: ReserveConfiguration: Incorrect Comment Attributes Bits 212-251 Hole to UnbackedMintCap Instead of DebtCeiling

This issue has been acknowledged by the team and fixed at commit `73e03ff`.

Description

In the `ReserveConfiguration.sol` file, a comment at line 49 incorrectly attributes the bit range from 212 to 251 to the `unbackedMintCap`, stating it is a leftover from pre-3.4 versions. However, the `unbackedMintCap` actually occupied bits 176-211, which were removed in version 3.4. The bits 212-251 were used for the `debtCeiling`, which was deprecated in version 3.7. The comment should correctly state that the bits are a leftover from the pre-3.7 `debtCeiling`. This incorrect attribution could mislead developers about the historical usage of these bits, potentially causing errors in future upgrades when reasoning about storage compatibility.

Attack Path

1. A developer reads the `ReserveConfiguration.sol` file and sees the comment about bits 212-251 being "left from pre 3.4 unbackedMintCap."
2. The developer incorrectly concludes these bits have been unused since version 3.4, when in fact they contained `debtCeiling` values until version 3.7.
3. In a future upgrade, the developer might assume these bits are guaranteed zero on all deployments that upgraded through version 3.4, when in reality deployments that had `debtCeiling` set between versions 3.4 and 3.7 will have non-zero values in bits 212-251.
4. If a new feature reuses bits 212-251 without clearing them, the stale `debtCeiling` values would be misinterpreted as the new feature's data.

Impact

This issue is primarily a documentation error with no direct functional impact on the current code behavior. However, it could lead to future misconfigurations if developers rely on the incorrect comment to determine the historical usage of bits 212-251. If these bits are reused in future upgrades without being properly zeroed, it could result in misinterpretation of data, potentially causing loss of function or incorrect behavior in the protocol.

Suggested Fix

Correct the comment to note the bits are from pre-3.7 debtCeiling removal.

```
-    //@notice there is an unoccupied hole of 40 bits from 212 to 251 left from pre
 ↪  3.4 unbackedMintCap
+    //@notice there is an unoccupied hole of 40 bits from 212 to 251 left from pre
 ↪  3.7 debtCeiling
```

Disclaimers

Sherlock AI does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.